

编 号	题 号	标 题	知 识 点	代 码 作 者
例 4-42	HDU 5992	Finding Hotels	kd-Tree: 最近邻点查询	陈锋
例 4-43	UVa 12939	Keep Fit	kd-Tree: 莫队	陈锋

4.8 可持久化数据结构

可持久化数据结构是一种比较新颖的、能解决复杂算法问题的设计思路。本节介绍了几种常见结构的可持久化版本的典型题目以及实现。

例 4-44 【可持久化线段树】区间第 K 小查询（ K -th Number, SPOJ MKTHNUM）

有 n ($1 \leq n \leq 100\,000$) 个数 $a_1, a_2, \dots, a_n$, 给出 m 次询问, 每次要询问一个区间 $[L, R]$ 中第 k 小的值是多少。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
const int MAXN = 100000 + 4;
struct Node;
typedef Node* PNode;
struct Node { // 权值线段树
    int count;
    PNode left, right;
    Node(int count = 0, PNode left = NULL, PNode right = NULL)
        : count(count), left(left), right(right) {}
    PNode insert(int l, int r, int w);
};

const PNode Null = new Node();
PNode Node::insert(int l, int r, int w) {
    if (l <= w && w < r) {
        if (l + 1 == r) return new Node(count + 1, Null, Null);
        int m = (l + r) / 2;
        return new Node(count + 1, left->insert(l, m, w), right->insert(m, r, w));
    }
    return this;
}

int query(PNode a, PNode b, int l, int r, int k) { // 二分查找逻辑
    if (l + 1 == r) return l;
    int m = (l + r) / 2;
    int count = a->left->count - b->left->count;
    if (count >= k) return query(a->left, b->left, l, m, k);
    return query(a->right, b->right, m, r, k - count);
}

int A[MAXN], RM[MAXN]; // 离散化
PNode VER[MAXN];

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    Null->left = Null->right = Null;
    int n, m, maxa = 0;
    cin >> n >> m;
    map<int, int> M;
    _for(i, 0, n) cin >> A[i], M[A[i]] = 0;
    for (map<int, int>::iterator p = M.begin(); p != M.end(); p++)
        p->second = maxa, RM[maxa] = p->first, maxa++;
    VER[0] = Null; // 权值线段树
    _for(i, 0, n) VER[i + 1] = VER[i]->insert(0, maxa, M[A[i]]);

    for (int i = 0, u, v, k; i < m; i++) {
        cin >> u >> v >> k;
        int ans = query(VER[v], VER[u - 1], 0, maxa, k);
        cout << RM[ans] << endl;
    }
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.11 节例题 53
注意：本题使用单独的 Null 结点来表示空结点，简化代码
同类题目：Count on a tree, SPOJ COT
*/

```

例4-45 【可持久化Trie】树上异或 (Tree, ACM/ICPC 2013南京, HDU 4757)

给出一棵包含 N ($N \leq 10^5$) 个点的树, 结点编号为 $1 \sim N$, 每个点 i 都给出其点权 a_i ($0 \leq a_i < 2^{16}$), 给出 m ($m \leq 10^5$) 个询问, 每个询问给出3个整数(x, y, v), 将 x 到 y 路径上的每个点与 v 进行异或运算, 求所得结果中的最大值。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i,a,b) for( int i=(a); i<(int)(b); ++i)
#define _rep(i,a,b) for( int i=(a); i<=(int)(b); ++i)
const int MAXH = 16, NN = 1e5 + 8, MM = NN * 32;
int A[NN], TC, Ver[NN];
vector<int> G[NN];
struct Trie { int ch[2], cnt; };
Trie B[MM];
int newTrie() {
    int c = TC++;
    fill_n(B[c].ch, 2, 0), B[c].cnt = 0;
    return c;
}

int insert(int p, int v, int dep) {
    int np = newTrie();
```

```

Trie &t = B[np], &t0 = B[p];
t = t0, t.cnt = t0.cnt + 1;
if (dep >= 0) {
    bool c = v & 1 << dep;
    t.ch[c] = insert(t0.ch[c], v, dep - 1);
}
return np;
}

int Fa[NN][MAXH + 1], D[NN]; // LCA
void dfs(int u, int f) {
    Fa[u][0] = f, D[u] = D[f] + 1;
    _rep(i, 1, MAXH) Fa[u][i] = Fa[Fa[u][i - 1]][i - 1];
    Ver[u] = insert(Ver[f], A[u], 15); // A[u] < 2^16
    for (auto v : G[u]) if (v != f) dfs(v, u);
}

int lca(int u, int v) {
    if (D[u] < D[v]) swap(u, v);
    int diff = D[u] - D[v];
    _rep(h, 0, MAXH) if (diff & (1 << h)) u = Fa[u][h];
    if (u == v) return u;
    for (int h = MAXH; h >= 0; h--)
        if (Fa[u][h] != Fa[v][h]) u = Fa[u][h], v = Fa[v][h];
    return Fa[u][0];
}

int query(int u, int v, int x) {
    int ans = 0, d = lca(u, v), ru = Ver[u], rv = Ver[v], rd = Ver[d];
    for (int i = 15; i >= 0; i--) { // x < 2^16, 从高位到低位遍历
        bool f = !(x & 1 << i);
        const Trie &tu = B[ru], &tv = B[rv], &td = B[rd];
        if (B[tu.ch[f]].cnt + B[tv.ch[f]].cnt > B[td.ch[f]].cnt * 2)
            ans |= 1 << i;
        else
            f = !f;
        ru = tu.ch[f], rv = tv.ch[f], rd = td.ch[f];
    }
    return max(ans, x ^ A[d]);
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    for (int n, m, u, v, x; cin >> n >> m; ) {
        for (int i = 1; i <= n; i++) cin >> A[i], G[i].clear();
        for (int i = 1; i < n; i++)
            cin >> u >> v, G[u].push_back(v), G[v].push_back(u);
        Ver[0] = TC = 0, newTrie(), dfs(1, 0);
        while (m--)
            cin >> u >> v >> x, printf("%d\n", query(u, v, x));
    }
}

```

```
}  
}  
/*
```

算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.11 节例题 55

注意：本题中每次修改结点前通过新建其拷贝来实现可持久化

同类题目：Xor Queries, CodeChef XRQRS

```
*/
```

例 4-46 【可持久化树状数组】网格监控 (Grid surveillance, IPSC 2011)

给定一个 4096×4096 的网格 G ，一开始每个格子都是 0。给出 q ($1 \leq q \leq 20\,000$) 个操作，其中修改操作将某个格子 (x, y) 增加一个特定的值 a ($0 \leq a \leq 100$)；查询操作给出 x_1, x_2, y_1, y_2, t ，要求计算第 t 次操作之前，以线段 $(x_1, y_1) - (x_2, y_2)$ 为对角线的子矩形中每个点的值之和。

注意：原题为了强制在线化，实际操作描述比较复杂，具体请参考原题描述[\(1\)](#)。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)

struct Item {
    int ver, c;
    bool operator<(const Item& i) const {
        if (ver != i.ver) return ver < i.ver;
        return c < i.c;
    }
};

template <int SZ>
struct BIT2D {
    vector<Item> C[SZ][SZ];
    int vals[SZ][SZ], version;
    BIT2D() { version = 0; }
    int lowbit(int x) { return x & (x ^ (x - 1)); }
    void add(int x, int y, int c) {
        int ver = ++version;
        vals[x][y] += c;
        for (int i = x; i < SZ; i += lowbit(i))
            for (int j = y; j < SZ; j += lowbit(j)) {
                vector<Item>& v = C[i][j];
                v.push_back({ver, v.empty() ? c : v.back().c + c});
            }
    }
};

// 版本 ver 中, [0,0] → [x,y] 区域的元素和
int sum(int x, int y, int ver) {

```

```

int ret = 0;
// 注意如何通过 Item 结构实现每个修改版本的可持久化存储
for (int i = x; i > 0; i -= lowbit(i))
    for (int j = y; j > 0; j -= lowbit(j)) {
        vector<Item>& v = C[i][j];
        vector<Item>::iterator it = // 查找 ver+1 之前的版本
            lower_bound(v.begin(), v.end(), (Item){ver + 1, 0});
        if (it != v.begin()) ret += (--it)->c;
    }
return ret;
}
};

const int DIM = 4096;
int XM(int x, int C) { return (x ^ C) % 4096 + 3; }
BIT2D<DIM + 16> S;

struct OP {
    int type, x1, x2, y1, y2, v;
    int exec(int c) {
        if (type == 1) {
            int x = XM(x1, c), y = XM(y1, c);
            S.add(x, y, v);
            return S.vals[x][y];
        }

        int _x1 = XM(x1, c), _x2 = XM(x2, c), _y1 = XM(y1, c), _y2 = XM(y2, c);
        int x1 = min(_x1, _x2), xr = max(_x1, _x2), y1 = min(_y1, _y2),
            yr = max(_y1, _y2);
        int ver; // 版本号
        if (v == 0)
            ver = S.version;
        else if (v > 0)
            ver = v;
        else if (v < 0)
            ver = max(S.version + v, 0);
        return S.sum(xr, yr, ver) + S.sum(x1 - 1, y1 - 1, ver) -
            S.sum(x1 - 1, yr, ver) - S.sum(xr, y1 - 1, ver);
    }
};

istream& operator>>(istream& is, OP& o) {
    is >> o.type;
    if (o.type == 1) is >> o.x1 >> o.y1;
    if (o.type == 2) is >> o.x1 >> o.x2 >> o.y1 >> o.y2;
    return is >> o.v;
}

int main() {

```

```

ios::sync_with_stdio(false), cin.tie(0);
int r, q, qc = 0;
cin >> r >> q;
vector<OP> ops(q);
for (auto& o : ops) {
    cin >> o;
    if (o.type == 2) qc++;
}
int c = 0, qi = 0;
_for(i, 0, r) for (auto& o : ops) {
    c = o.exec(c);
    if (o.type == 2) {
        if (qi + 20000 >= r * qc) cout << c << endl;
        ++qi;
    }
}
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.11 节例题 56
同类题目: To the Moon, HDU 4348
*/

```

例4-47 【可持久化Treap】自带版本控制功能的IDE (Version Controlled IDE, ACM/ICPC Hatyai 2012, UVa 12538)

编写一个支持查询历史记录功能的编辑器，支持以下3种操作。

- ☐ 1 p s : 在位置 p 前插入字符串 s 。
- ☐ 2 p c : 从位置 p 开始删除 c 个字符。
- ☐ 3 v p c : 打印版本 v 中从位置 p 开始的 c 个字符。

缓冲区一开始是空串，对应版本号0。每次执行操作1或操作2之后，版本号加1。每个查询回答之后，才能读下一个查询。操作数 $n \leq 50\,000$ ，插入串总长不超过1MB，输出总长保证不超过200KB。

【代码实现（基于可以分裂合并的Treap）】


```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (decltype(b) i = (a); i < (b); ++i)
const int MAXN = (1 << 23), MAXQ = 50000 + 4;
struct Node;
typedef Node *PNode;
PNode Null, VER[MAXN];
struct Node {
    PNode left, right;
    char label; // 用户自定义 label
    int key, sz;
    Node(char c = 0, int s = 1) : label(c), sz(s) {
        left = right = Null, key = rand();
    }
};

```

```

    }
    PNode update() {
        sz = 1 + left->sz + right->sz;
        return this;
    }
};

Node Nodes[MAXN];
struct Treap {
    int bufIdx = 0, d;
    PNode copyOf(PNode u) {
        if (u == Null) return u;
        PNode ret = &Nodes[bufIdx++];
        *ret = *u;
        return ret;
    }
    PNode merge(PNode a, PNode b) {
        if (a == Null) return copyOf(b);
        if (b == Null) return copyOf(a);
        PNode ret;
        if (a->key < b->key)
            ret = copyOf(a), ret->right = merge(a->right, b);
        else
            ret = copyOf(b), ret->left = merge(a, b->left);
        return ret->update();
    }
}

void split(PNode pn, PNode &l, PNode &r, const int k) {
    int psz = pn->sz, plsz = pn->left->sz;
    if (k == 0)
        l = Null, r = copyOf(pn);
    else if (psz <= k)
        l = copyOf(pn), r = Null;
    else if (plsz >= k)
        r = copyOf(pn), split(pn->left, l, r->left, k), r->update();
    else
        l = copyOf(pn), split(pn->right, l->right, r, k - plsz - 1), l->update();
}

PNode build(int l, int r, const char *s) {
    if (l > r) return Null;
    int m = (l + r) / 2;
    Node u(s[m]);
    PNode a = copyOf(&u), p = build(l, m - 1, s), q = build(m + 1, r, s);
    p = merge(p, a), a = merge(p, q);
    return a->update();
}

PNode insert(const PNode ver, int pos, const char *s) {
    PNode p, q, r = build(0, strlen(s) - 1, s);
    split(ver, p, q, pos);
    return merge(merge(p, r), q);
}

```

```

}
PNode remove(PNode ver, int pos, int n) {
    PNode p, q, r;
    split(ver, p, q, pos - 1), split(q, q, r, n);
    return merge(p, r);
}
void print(PNode ver) {
    if (ver == Null) return;
    print(ver->left), d += (ver->label == 'c');
    putchar(ver->label);
    print(ver->right);
}
void debugPrint(PNode pn) {
    if (pn == Null) return;
    debugPrint(pn->left), putchar(pn->label), debugPrint(pn->right);
}
void traversal(PNode pn, int pos, int n) {
    PNode p, q, r;
    split(pn, p, q, pos - 1), split(q, q, r, n), print(q);
}
void init() { bufIdx = 0, d = 0, Null = &Nodes[bufIdx++], Null->sz = 0; }
};
Treap tree;
int main() {
    int n, opt, v, p, c, ver = 0;
    scanf("%d", &n), tree.init();
    char s[128];
    VER[0] = Null;
    _for(i, 0, n) {
        scanf("%d", &opt);
        switch (opt) {
            case 1:
                scanf("%d %s", &p, s), p -= tree.d;
                VER[ver + 1] = tree.insert(VER[ver], p, s), ver++;
                break;
            case 2:
                scanf("%d %d", &p, &c), p -= tree.d, c -= tree.d;
                VER[ver + 1] = tree.remove(VER[ver], p, c), ver++;
                break;
            case 3:
                scanf("%d%d%d", &v, &p, &c), v -= tree.d, p -= tree.d, c -= tree.d;
                tree.traversal(VER[v], p, c), puts("");
                break;
            default:
                break;
        }
    }
    return 0;
}

```

```
/*
```

```
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.11 节例题 57
```

```
同类题目：SuperMemo, POJ 3580
```

```
*/
```

还有一种基于STL中rope的实现方法，rope就是一个用块状链表实现的“重型”string（然而它也可以保存int或其他类型），它属于STL扩展。代码中，crope即rope<char>。

【代码实现（基于块状链表rope的实现）】

```

// 陈锋
#include <bits/stdc++.h>

#include <ext/rope>
using namespace std;
using namespace __gnu_cxx;
crope ro, version[50100];

int main() {
    int n, d = 0, ver = 1;
    string buf;
    cin >> n;
    for (int i = 0, opt, p, c, v; i < n; i++) {
        cin >> opt;
        switch (opt) {
            case 1:
                cin >> p >> buf, p -= d;
                ro.insert(p, buf.c_str()), version[ver++] = ro; // 保留历史版本
                break;
            case 2:
                cin >> p >> c, p -= d, c -= d;
                ro.erase(p - 1, c), version[ver++] = ro; // 保留历史版本
                break;
            default:
                cin >> v >> p >> c;
                v -= d, p -= d, c -= d;
                const crope& tmp = version[v].substr(p - 1, c);
                for (size_t i = 0; i < tmp.size(); i++) {
                    char c = tmp[i];
                    d += (c == 'c'), cout << c;
                }
                cout << endl;
                break;
        }
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.11 节例题 57
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表4-10所示。

表4-10 可持久化数据结构例题归纳